

# Forecasting Staten Island Railway On-Time Performance with Explainable and Uncertainty-Aware Machine Learning

**Course:** ANLY 530 Final Project Report

**Author:** Chinmay Naringrekar

**Repository:** Staten Island OTP Forecasting

## 1. Introduction and Motivation

The Staten Island Railway (SIR) is the only rapid-transit line serving Staten Island, and for many residents it is the primary link to the Staten Island Ferry and, from there, to Manhattan. On-time performance (OTP) is the Metropolitan Transportation Authority's (MTA) headline reliability measure for the line: the share of scheduled trips that arrive at their terminal within the on-time threshold. The MTA publishes SIR OTP monthly through its open data program, but this reporting is retrospective. It describes how the railway performed last month; it says nothing about how it is likely to perform next month.

That gap motivates this project. If monthly OTP can be forecast with useful accuracy even one month ahead, planners gain lead time to schedule maintenance, adjust staffing, and communicate expectations. A preliminary examination of the published series suggested analytics could help: monthly OTP shows persistent momentum (good and bad months cluster), visible seasonal structure, and a long history (2006 to 2026): exactly the conditions under which supervised learning on lagged features tends to work. The shortcoming of the current practice is not a lack of data but a lack of forward-looking, interpretable use of it. This project builds a forecasting system that predicts next-month OTP, explains its predictions with SHAP, quantifies uncertainty with prediction intervals, and delivers the results through an interactive dashboard.

## 2. Problem Definition and Research Question

The problem is short-horizon forecasting of a monthly service-reliability metric. Formally: given the operational history of the Staten Island Railway through month  $t$ , predict the 7-Day on-time performance for month  $t + 1$ .

The main research question, formalized in the project's methodology phase (notebook 10), is:

*Can gradient-boosted tree models using lag-based, rolling-window, calendar, and operational features outperform traditional statistical time-series baselines such as SARIMA and Prophet in forecasting short-horizon commuter rail on-time performance, while providing interpretable operational insights through SHAP analysis?*

Supporting questions address which features contribute most to prediction, how stable performance is under time-aware cross-validation, and whether prediction intervals can communicate forecast uncertainty well enough to support operational decisions.

The target variable throughout is **next-month 7-Day On-Time Performance (without boat)**: the On-Time Performance column of the MTA dataset, filtered to the 7-Day Day Time category. The dataset also reports an On-Time Performance (With Boat) variant that penalizes trains missing the ferry connection; that variant is not used as the target and is reserved for future comparison.

### 3. Related Work and Background

Rail on-time performance forecasting sits at the intersection of classical time-series analysis and applied machine learning. Statistical approaches such as seasonal ARIMA (SARIMA) have long been the default for monthly transit indicators because they model trend, autocorrelation, and seasonality directly. Prophet (Taylor & Letham, 2018) offers a decomposable trend-plus-seasonality model that is popular for business time series with similar characteristics. More recently, gradient-boosted tree ensembles, particularly XGBoost (Chen & Guestrin, 2016), have become the standard for structured tabular prediction, and a common pattern in applied forecasting is to recast a time series as a supervised learning problem using lagged and rolling-window features, which is the approach taken here.

Two further strands of work shape the project. SHAP (Lundberg & Lee, 2017) provides additive, game-theoretic attributions that make tree-ensemble predictions interpretable, which matters in an operational setting where a forecast is only actionable if planners can see what is driving it. And the forecasting literature consistently warns that machine learning models must be benchmarked against naive and smoothing baselines under chronologically honest evaluation, since random train/test splits leak future information. This project adopts both disciplines: every headline result is benchmarked against naive, moving-average, SARIMA, and Prophet baselines, and all headline evaluation preserves chronological order.

This report does not claim a comprehensive literature survey of transit OTP prediction; the related-work review is scoped to the methods actually used.

### 4. Data Description

The data source is the MTA's public Staten Island Railway On-Time Performance dataset (data/raw/MTA\_Staten\_Island\_Railway\_On-Time\_Performance.xlsx), cleaned into data/raw/cleaned\_staten\_island\_otp.csv.

- **Coverage:** monthly observations from January 2006 through January 2026.
- **Size:** 1,205 rows across five Day Time reporting categories (Weekday, AM Rush, PM Rush, Weekend, and 7-Day), of which 241 monthly rows belong to the 7-Day category used for the main research comparison.
- **Columns:** Month, Day Time, Delayed Trains, On-Time Trips, On-Time Performance, the three (With Boat) variants of those counts, Scheduled Trips, Incomplete Trips, Trip Complete Percentage, and derived calendar fields (Year, Month\_Number, Month\_Name).

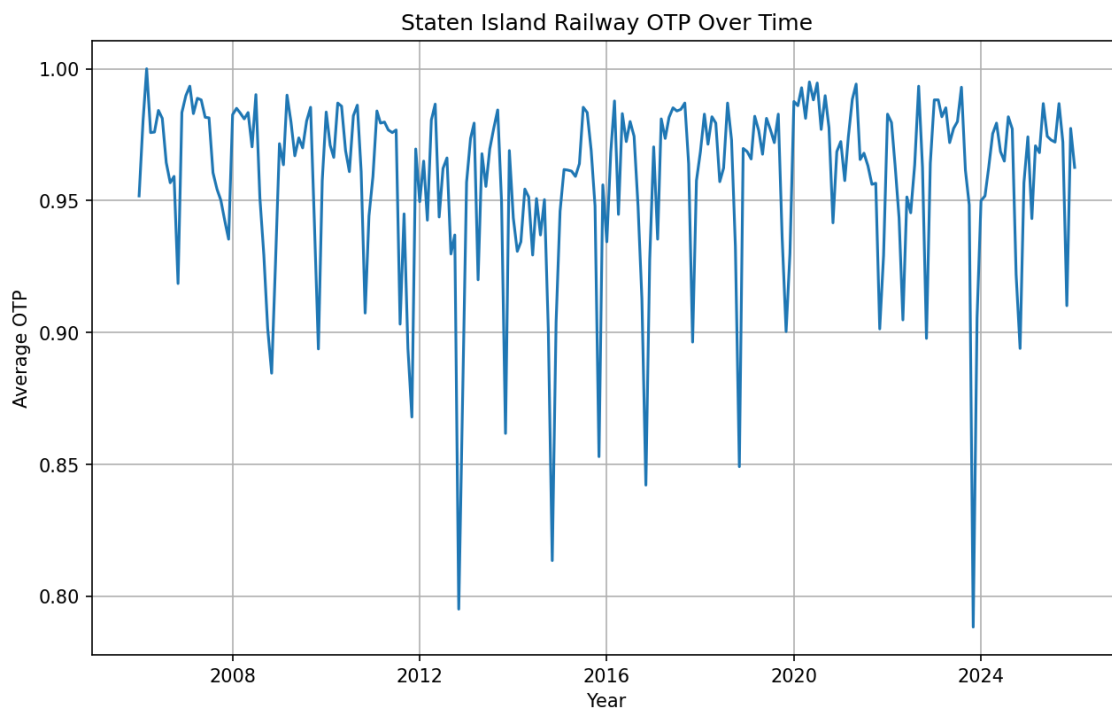
Data rejected or excluded: the (With Boat) performance variant is excluded from the target definition (Section 2); Incomplete Trips and Trip Complete Percentage are missing in early years and are

used only where available. No external data (weather, incidents, ridership) is included in the current version; that is future work, not completed work (Section 15).

## 5. Exploratory Data Analysis and Preprocessing

Exploratory analysis (notebook 02) examined the OTP trend over time, the delayed-trains trend, and seasonal structure. Three observations shaped the modeling:

1. **Momentum.** Monthly OTP is strongly autocorrelated; a month's OTP is close to recent months' OTP far more often than not. This motivated lag and rolling-average features.
2. **Seasonality.** OTP varies systematically across the calendar, with recurring within-year patterns. This motivated month, quarter, and season features. SHAP later bears this out, ranking the month number as the single most influential feature (Section 11).
3. **Regime changes.** The twenty-year series spans distinct operational eras, including the pandemic period, so any single train/test split risks measuring era effects rather than model quality. This motivated the time-series cross-validation of Phase 14.



**Figure 1.** Historical Staten Island Railway on-time performance, 2006–2026.

Preprocessing steps were modest by design: parsing month timestamps, deriving calendar fields, retaining the Day Time category structure, and dropping rows made incomplete by feature construction (next paragraph). Readiness for machine learning was established by confirming that after feature engineering no nulls remained in the model table and that all features are numeric (categorical fields one-hot encoded).

## 6. Feature Engineering

Notebook 03 converts the time series into a supervised learning problem. For each Day Time group, the target Next\_Month\_OTP is the On-Time Performance value shifted one month backward, so each row pairs the current month's operational state with the following month's outcome. Features include:

- **Lag features:** OTP lags 1, 2, and 3 months; delayed-trains lag 1.
- **Rolling windows:** 3- and 6-month rolling mean OTP; 3-month rolling mean delayed trains.
- **Operational load:** delayed trains, on-time trips, scheduled trips, delay rate, trip-completion percentage.
- **Calendar:** year, month number, quarter, plus one-hot encoded season and Day Time categories.

Rows lacking a full lag/rolling history or a next-month target are dropped, yielding a model table of **1,170 rows and 26 columns**. Because the target is a one-month-forward shift, the feature table necessarily ends at 2025-12: the final raw month (2026-01) has no observed "next month" and drops out of the supervised dataset.

## 7. Technical Approach

The project proceeds in phases, each in its own notebook (01 through 10), with artifacts written to outputs/:

1. Load and clean the raw MTA data (notebook 01) and explore it (notebook 02).
2. Engineer the supervised feature table (notebook 03).
3. Train and compare candidate regressors (Linear Regression, Random Forest, XGBoost) on a chronological 80/20 split, and select a primary model (notebook 04).
4. Interpret the selected model with SHAP (notebook 05).
5. Generate recursive multi-month forecasts, feeding each prediction back in as a lag feature for horizons of 3–12 months (notebook 06).
6. Benchmark the selected model against forecasting baselines under a fair, leakage-free protocol (**Phase 13**, notebook 07).
7. Stress-test stability with 5-fold time-series cross-validation (**Phase 14**, notebook 08).
8. Add residual-based prediction intervals (**Phase 15**, notebook 09).
9. Formalize the research question and methodology (Phase 16, notebook 10).
10. Serve results through a Streamlit dashboard (app/streamlit\_app.py).

The defining methodological choice is chronological honesty: every split used for a reported claim preserves time order, and the Phase 13 comparison retrains XGBoost using only data available before the test period, so the machine learning model enjoys no information advantage over the statistical baselines.

## 8. Modeling Methods

Three supervised regressors were trained on the feature table (notebook 04) using a chronological 80/20 split, with OTP on its native 0–1 proportion scale:

| Model             | MAE    | RMSE   | R <sup>2</sup> |
|-------------------|--------|--------|----------------|
| Linear Regression | 0.0305 | 0.0487 | 0.119          |
| Random Forest     | 0.0311 | 0.0476 | 0.159          |
| XGBoost           | 0.0319 | 0.0472 | 0.174          |

Linear regression served as the simplicity baseline; random forest tested for non-linear threshold effects; XGBoost was configured with 300 estimators, learning rate 0.05, and maximum depth 5. The three models performed within about 0.001–0.002 MAE of one another, but XGBoost achieved the best RMSE and R<sup>2</sup>, and it supports efficient SHAP interpretation, so it was selected as the primary model and saved to `models/xgboost_otp_model.pkl`.

For the Phase 13 research comparison, the forecasting baselines were: a naive last-value forecast, 3- and 6-month moving averages, SARIMA (via statsmodels), and Prophet. These span the range from trivial to genuinely competitive statistical methods and provide the comparison the research question requires.

## 9. Validation and Evaluation Metrics

Three complementary evaluation designs guard against overclaiming:

- **Fair hold-out comparison (Phase 13).** Training data runs through July 2025; the test period is August 2025 to January 2026; target restricted to the 7-Day series; XGBoost retrained from scratch on pre-test data only.
- **Time-series cross-validation (Phase 14).** `TimeSeriesSplit` with 5 chronological folds over the 2006–2026 history, 39 test rows per fold, training always strictly preceding testing. This tests whether Phase 13's result generalizes across eras.
- **Interval calibration (Phase 15).** Residuals pooled from the Phase 14 folds define empirical 80% and 90% prediction intervals (10th/90th and 5th/95th residual percentiles), evaluated by observed coverage.

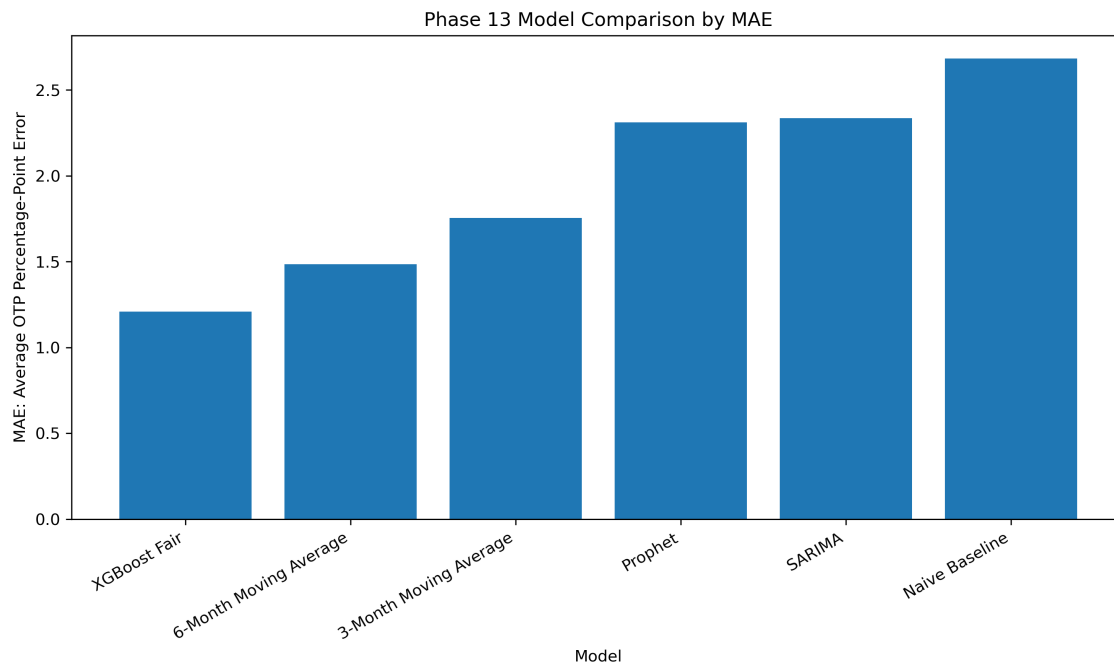
Metrics are MAE, RMSE, R<sup>2</sup>, and MAPE. MAE is the primary metric: it reads directly in OTP percentage points and is robust to the occasional disrupted month; RMSE exposes large errors; R<sup>2</sup> is reported but interpreted cautiously because short monthly series produce unstable R<sup>2</sup>; MAPE aids comparability. Phases 13–15 report OTP in percentage points (0–100), while the notebook 04 selection step used the 0–1 proportion scale; the two are noted separately to avoid unit confusion.

## 10. Results and Findings

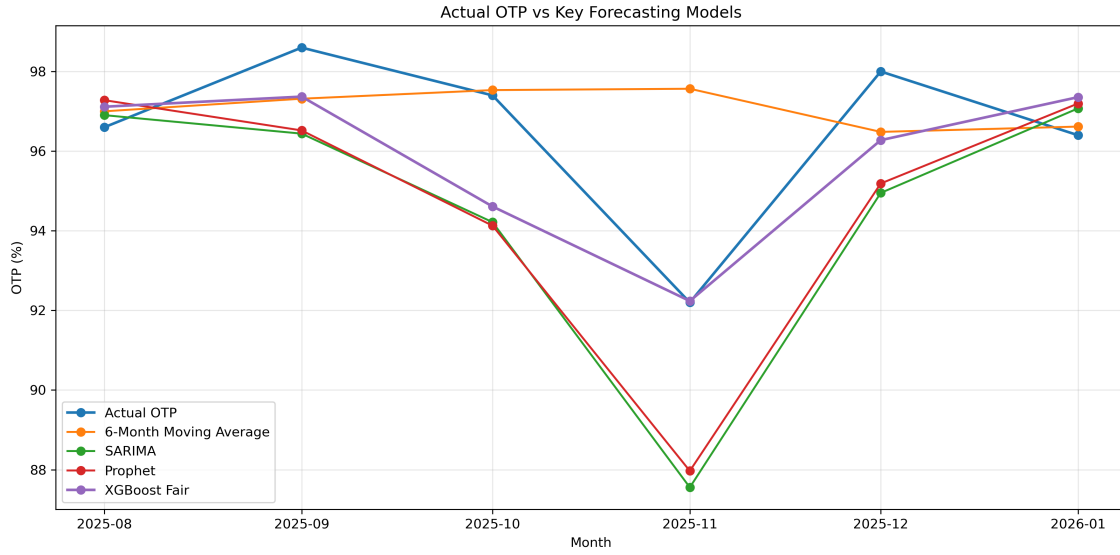
**Phase 13: fair baseline comparison** (full table in outputs/reports/phase13\_model\_comparison.csv):

| Model                  | MAE           | RMSE          | R <sup>2</sup> | MAPE          |
|------------------------|---------------|---------------|----------------|---------------|
| <b>XGBoost Fair</b>    | <b>1.2087</b> | <b>1.4974</b> | <b>0.4821</b>  | <b>1.2394</b> |
| 6-Month Moving Average | 1.4861        | 2.3443        | -0.2695        | 1.5743        |
| 3-Month Moving Average | 1.7556        | 2.4616        | -0.3998        | 1.8507        |
| Prophet                | 2.3118        | 2.6438        | -0.6147        | 2.4095        |
| SARIMA                 | 2.3357        | 2.7753        | -0.7793        | 2.4368        |
| Naive Baseline         | 2.6833        | 3.3855        | -1.6477        | 2.7982        |

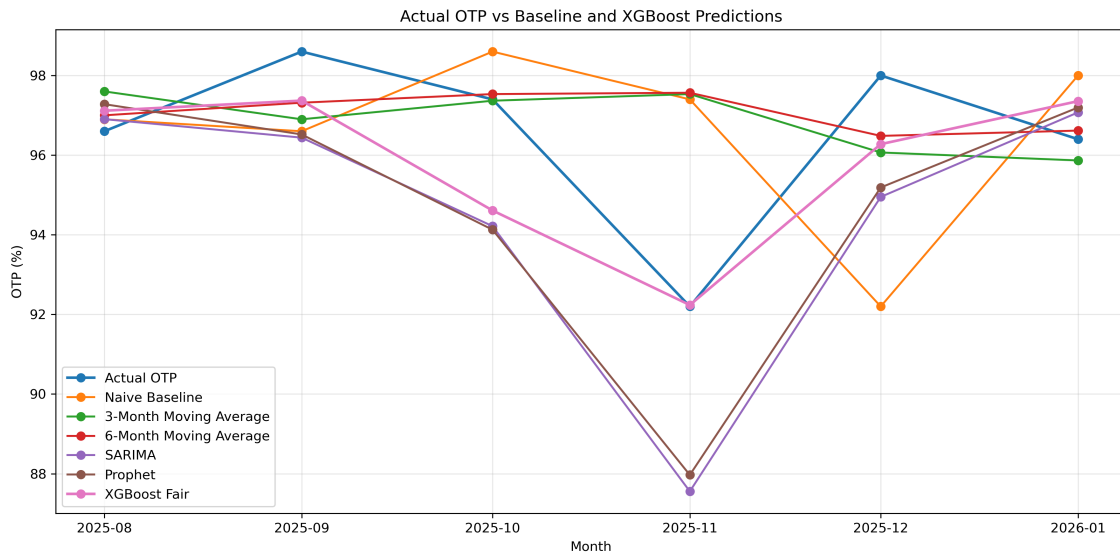
The fairly retrained XGBoost model reduced MAE by roughly **18.7%** relative to the best simple baseline (the 6-month moving average) and was the only model with positive R<sup>2</sup> on the test window. Notably, the statistical models (SARIMA, Prophet) trailed even the moving averages on this short horizon, a finding consistent with the strong month-to-month momentum in the series, which smoothing captures cheaply but which trend/seasonality decompositions can over-model.



**Figure 2.** Phase 13 model comparison by MAE.

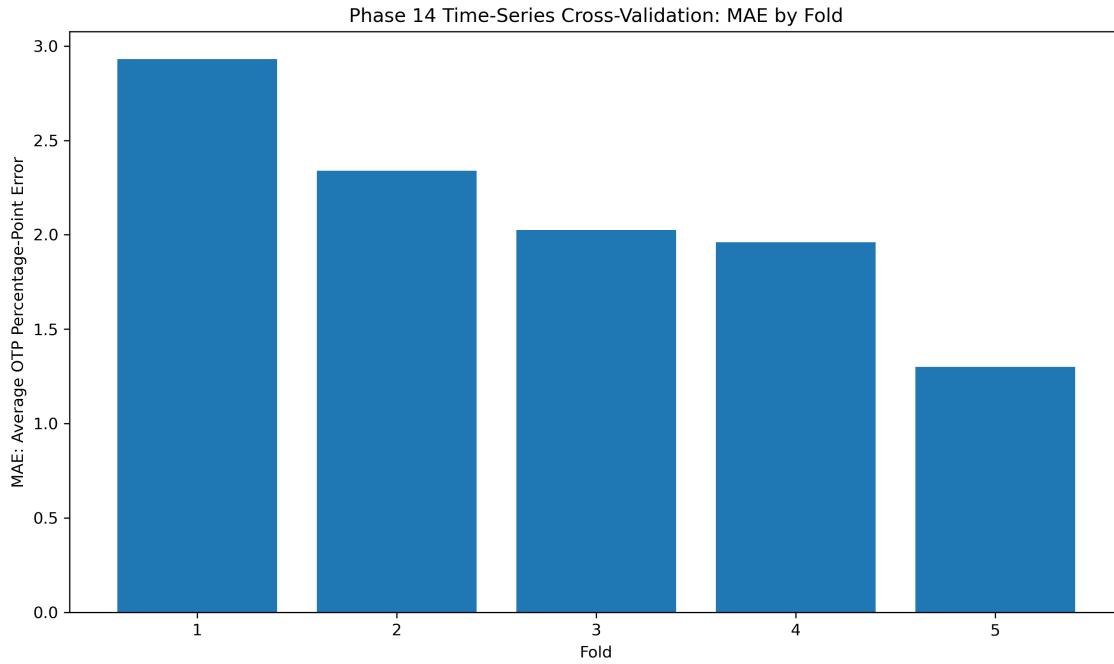


**Figure 3.** Actual OTP versus key model predictions over the Phase 13 test window.

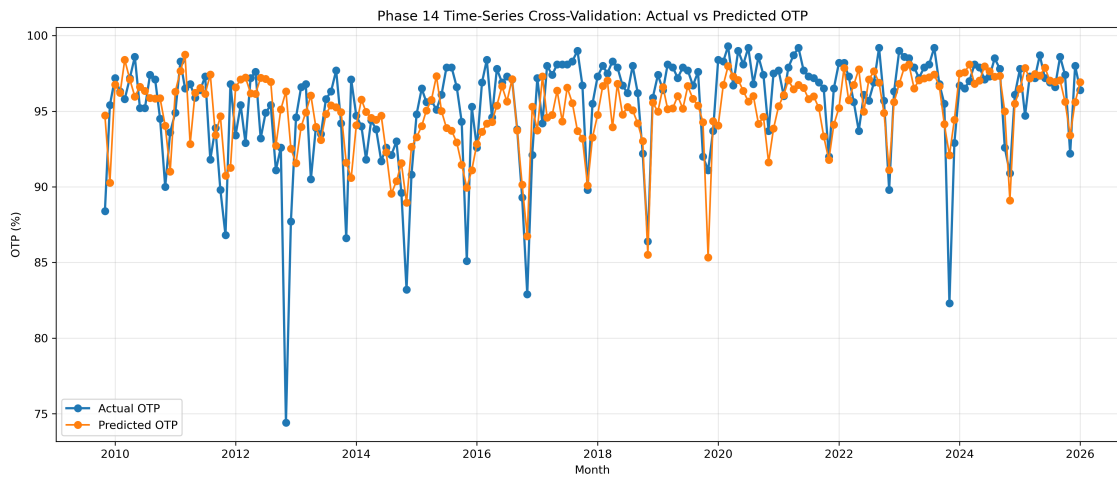


**Figure 4.** Actual OTP versus baseline model predictions (Phase 13).

**Phase 14:** **time-series** **cross-validation**  
(outputs/reports/phase14\_timeseries\_cv\_summary.csv): across five chronological folds the model averaged **MAE  $2.1107 \pm 0.5936$ , RMSE 2.8548,  $R^2$  0.1820, MAPE 2.2677%**. Performance improved almost monotonically with training history: the weakest fold was Fold 1 (MAE 2.93, trained on only 39 rows from 2006–2009) and the strongest was Fold 5 (MAE 1.30, trained on 195 rows through 2022). The CV average is substantially worse than the Phase 13 single-window result, which is itself a finding: headline accuracy depends on the evaluation era, and the single-split number should be read as an optimistic-recent-era estimate rather than a universal one.

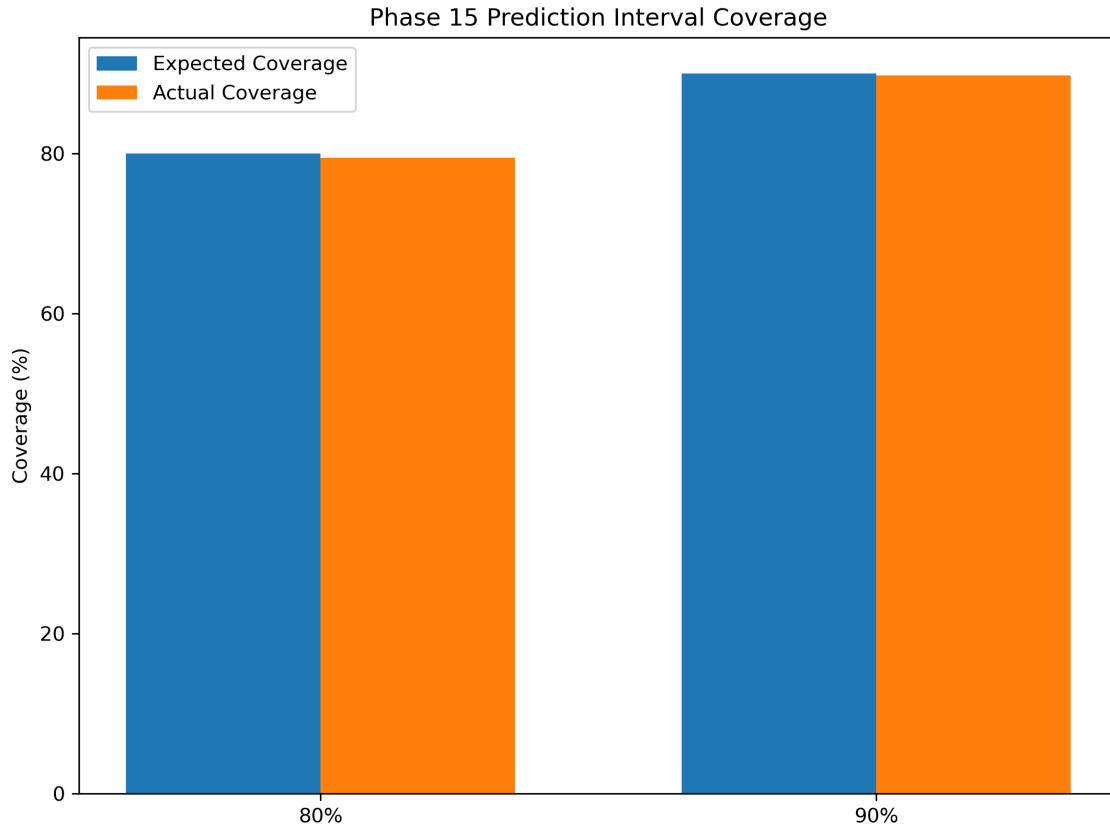


**Figure 5.** MAE by chronological cross-validation fold (Phase 14).

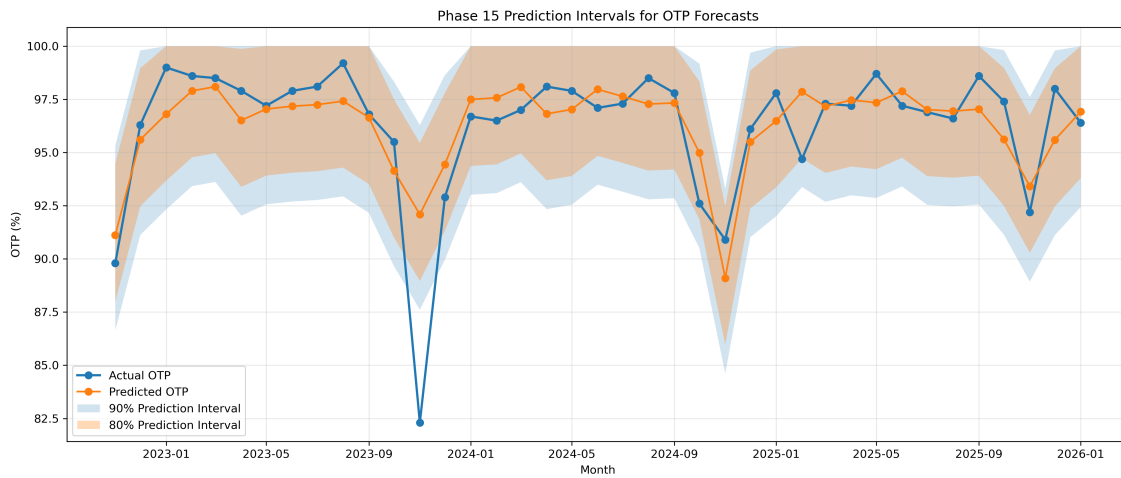


**Figure 6.** Actual versus predicted OTP across cross-validation folds (Phase 14).

**Phase 15: prediction intervals**  
(outputs/reports/phase15\_prediction\_interval\_summary.csv): the residual-based 80% interval achieved **79.49%** observed coverage (average width 6.30 OTP points) and the 90% interval achieved **89.74%** coverage (width 8.18 points). Both are within about half a percentage point of their nominal levels, indicating well-calibrated uncertainty estimates.



**Figure 7.** Observed versus expected prediction-interval coverage (Phase 15).



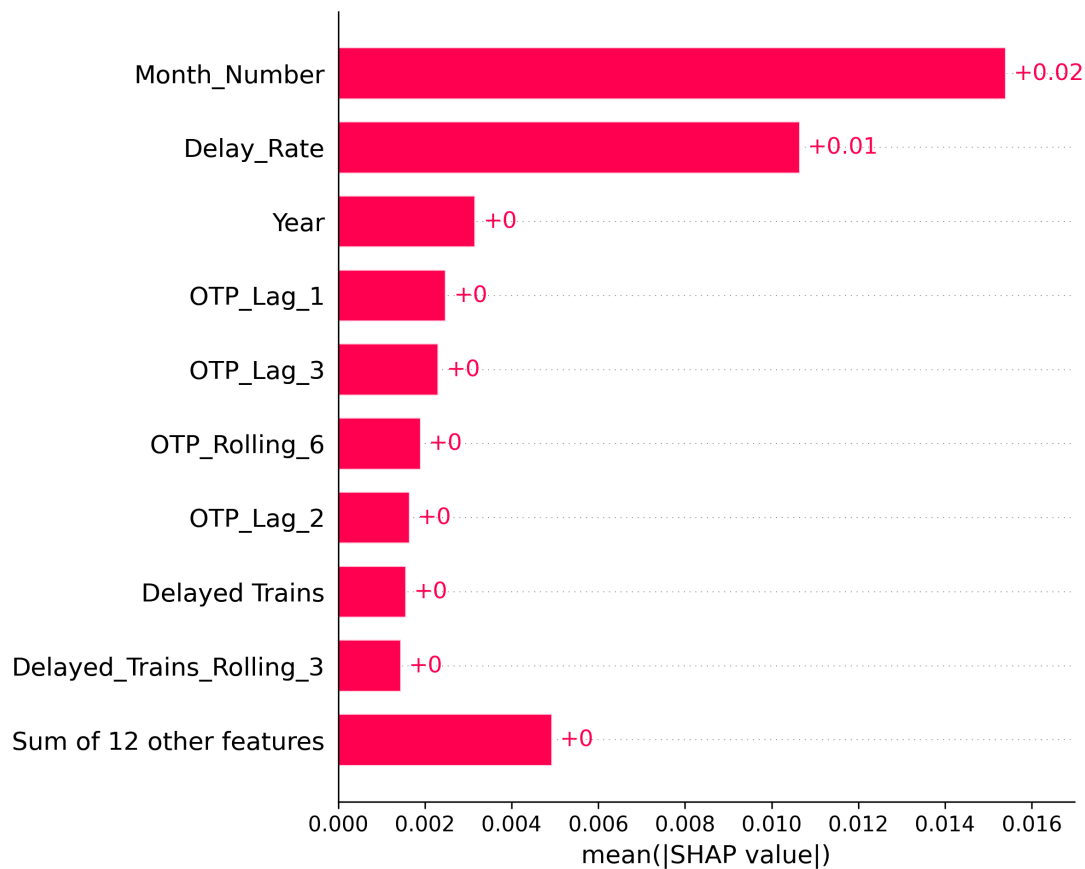
**Figure 8.** OTP predictions with 80% and 90% prediction intervals (Phase 15).

Taken together, the results answer the research question affirmatively but with an honest qualifier: the gradient-boosted model does outperform statistical and simple baselines on the fair recent-era comparison, its advantage is real but era-dependent under cross-validation, and its uncertainty can be quantified with near-nominal calibration.

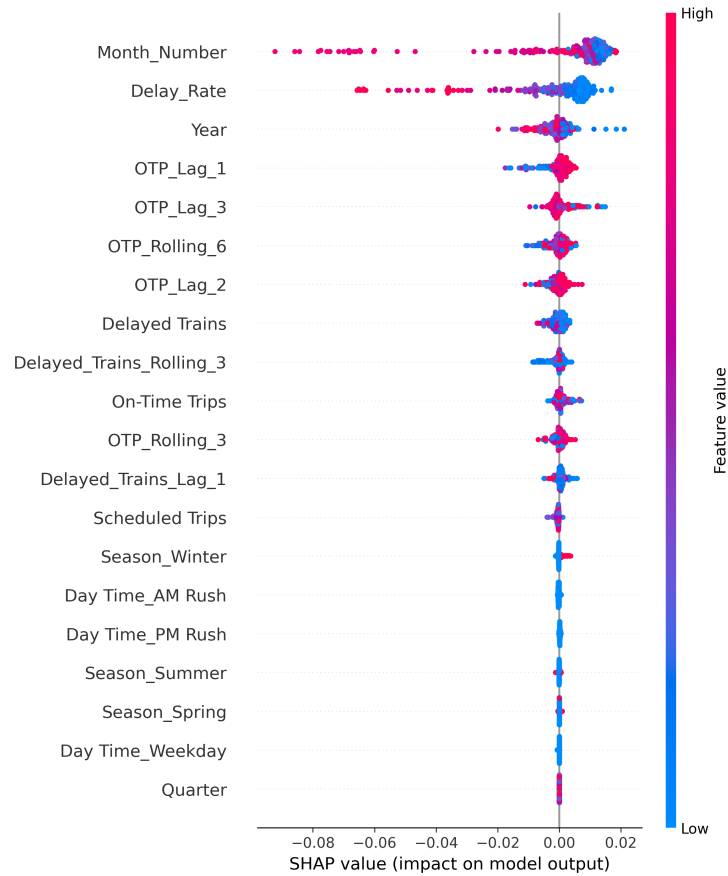
## 11. Explainability with SHAP

SHAP analysis of the trained XGBoost model (notebook 05; outputs/figures/shap\_summary.png and outputs/figures/shap\_bar.png) ranks the most influential features as:

1. **Month\_Number**, by a wide margin, confirming that seasonality is the dominant systematic driver of next-month OTP.
2. **Delay\_Rate**: the current month's delay burden carries forward.
3. **Year**, capturing long-run era effects.
4. **OTP lags and rolling means** (OTP\_Lag\_1, OTP\_Lag\_3, OTP\_Rolling\_6, OTP\_Lag\_2): recent OTP momentum.
5. **Delayed Trains** and Delayed\_Trains\_Rolling\_3: raw operational delay volume.



**Figure 9.** Mean absolute SHAP value by feature.



**Figure 10.** SHAP summary plot for the XGBoost model.

The picture is operationally coherent: where the railway is in the calendar, how much delay it is currently carrying, and how it has trended recently jointly determine the forecast. This moves the model from a black box to something a planner can interrogate; the dashboard's explainability section asks not only *what* the model predicts but *why*.

## 12. Prediction Intervals and Uncertainty

Point forecasts alone overstate certainty. Phase 15 attaches empirical intervals to each forecast by taking percentiles of the chronologically honest residuals from Phase 14: the 10th/90th percentiles ( $-3.13$ ,  $+3.36$  OTP points) bound the 80% interval, and the 5th/95th percentiles ( $-4.48$ ,  $+4.20$ ) bound the 90% interval. Because the intervals derive from out-of-sample residuals rather than in-sample fit, their near-nominal observed coverage (79.49% and 89.74%) is a meaningful calibration result, not an artifact. For a decision-maker the practical reading is: a next-month forecast of, say, 96% OTP should be treated as "roughly 93 to 99" at 80% confidence.

## 13. Dashboard and Decision Support

The Streamlit application (`app/streamlit_app.py`) packages the analysis for non-technical users. It provides executive KPI cards (average, best, and worst OTP and average delay rate), an interactive

historical trend with Day Time filtering, actual-versus-predicted comparison, recursive future forecasting with a user-selected 3–12 month horizon, and SHAP-based explanation of prediction drivers. Forecasted months are classified into operational risk levels of Low (at or above 95% OTP), Medium (90 to 95%), and High (below 90%), with a plain-language outlook message, translating model output into the vocabulary of service planning.

## 14. Limitations

- **Small effective sample.** The headline 7-Day series has 241 monthly observations; the Phase 13 test window is six months. Metric estimates carry wide sampling error, and  $R^2$  in particular is unstable.
- **Era dependence.** Cross-validated MAE (2.11) is materially worse than the recent-era hold-out MAE (1.21). The model is most trustworthy where training history is long and the operating regime resembles the recent past; structural breaks such as the pandemic degrade it.
- **No exogenous drivers.** Weather, incidents, maintenance schedules, and ridership are absent. The model can only extrapolate internal momentum and seasonality; it cannot anticipate externally caused disruptions.
- **Recursive forecast error accumulation.** Multi-month forecasts feed predictions back as inputs, so uncertainty compounds with horizon; the calibrated intervals of Phase 15 apply to the one-step forecast.
- **Single line, single metric.** Results are specific to the SIR 7-Day OTP series and may not transfer to other lines or reliability metrics without revalidation.

## 15. Future Work

Planned extensions, none of which are part of the completed work reported above: (1) weather feature enhancement, joining precipitation, snowfall, temperature, and wind to the monthly series with a controlled comparison against the base model; (2) incident, maintenance, and ridership covariates as they become available in open data; (3) refinement of the OTP risk classification, including uncertainty-aware risk bands that combine the point forecast with the Phase 15 intervals; (4) comparison of alternative targets, including the With Boat OTP variant and peak-period series; and (5) a dashboard upgrade that surfaces the Phase 13–15 research results directly.

## 16. Replicability

The repository is organized for independent reproduction. `requirements.txt` lists all dependencies (including `statsmodels` and `prophet` for the baselines). The README's *How to Run* section documents the notebook execution order (01 through 10, with 01 to 03 required first because they produce the cleaned data and feature table) and the dashboard launch command (`streamlit run app/streamlit_app.py`). All reported numbers are persisted as artifacts under `outputs/reports/` (CSV summaries and findings text files) and all figures under `outputs/figures/`, so a reader can verify every claim in this report against saved output without rerunning anything. One reproducibility

caveat: the raw MTA dataset is a point-in-time export; a fresh download may extend beyond January 2026 and shift results slightly. Within the committed data, someone unfamiliar with the project can replicate it by following the README in order.

## 17. Conclusion

This project set out to determine whether a gradient-boosted tree model with engineered temporal features could out-forecast traditional statistical baselines on Staten Island Railway on-time performance, and whether its predictions could be made interpretable and uncertainty-aware. Under a leakage-free comparison, XGBoost reduced next-month forecast error by about 18.7% against the best simple baseline and decisively outperformed SARIMA and Prophet. Time-series cross-validation tempered the headline: accuracy varies by era and improves with training history. SHAP attributed the model's skill to seasonality, current delay burden, and OTP momentum (drivers a transit planner would recognize), and residual-based prediction intervals achieved near-nominal coverage, turning point forecasts into defensible ranges. The finished product is a small, reproducible decision-support system: forecast, explanation, uncertainty, and risk framing, delivered through an interactive dashboard and documented for replication.

## References

- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45.
- Metropolitan Transportation Authority. *MTA Staten Island Railway On-Time Performance* [Data set]. MTA Open Data.